MILESTONE #3 PROJECT REPORT

FitWise – Workout Tracking Web App

Author: Emmanuel Dairo

CS499 Senior Seminar

Milestone #3 Progress Report

Date: April 14, 2026

1. Introduction

FitWise is a full-stack workout tracking and training insights system designed to help users log workouts and analyze performance over time. The application allows users to record exercises with sets, reps, and weight while also providing analytics such as total volume, personal records, and training trends.

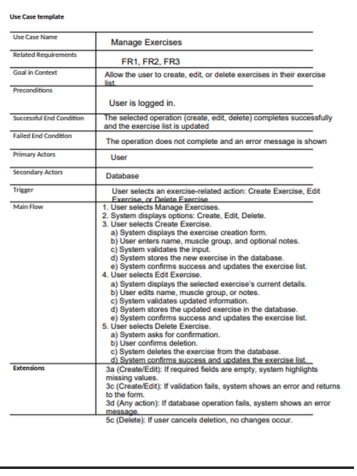
Unlike basic tracking applications, FitWise transforms workout data into meaningful insights, helping users understand consistency, progression, and muscle group distribution.

2. First Semester Results (Specification, Design, Analysis)

During the first phase of the project, the system design, requirements, and architecture were developed. This included defining functional requirements, creating UML diagrams, and designing the interaction between system components.

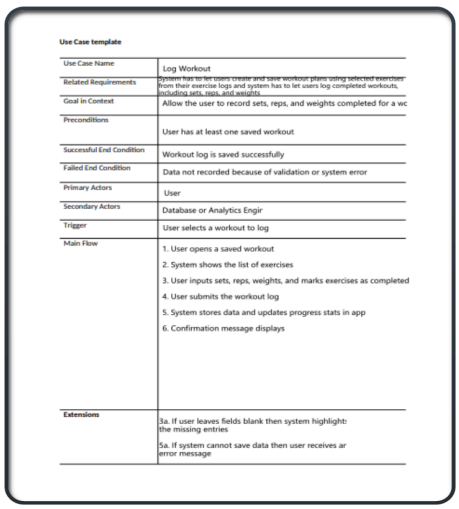
The system was designed as a full-stack web application using a React frontend, FastAPI backend, and SQLite database. The architecture followed a layered approach with clear separation between the user interface, business logic, and data storage.

The focus of this phase was establishing a stable foundation for workout tracking, including CRUD operations, workout logging, and structured data storage.



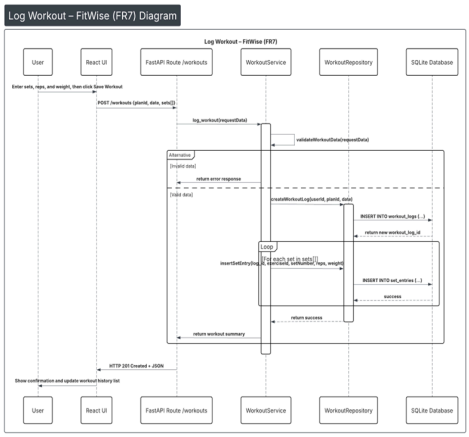
Use Case template

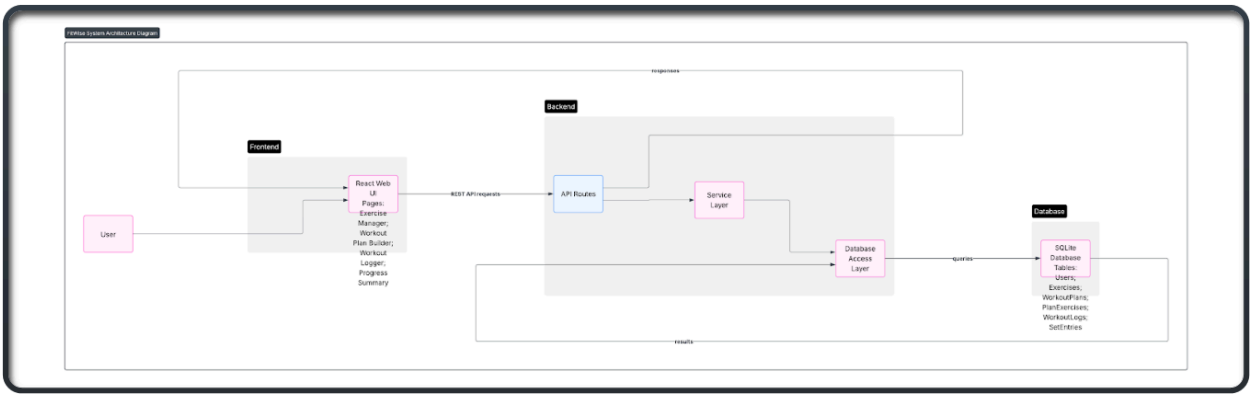
Use Case Name	Manage Exercises
Related Requirements	FR1, FR2, FR3
Goal in Context	Allow the user to create, edit, or delete exercises in their exercise list.
Preconditions	User is logged in.
Successful End Condition	The selected operation (create, edit, delete) completes successfully and the exercise list is updated
Failed End Condition	The operation does not complete and an error message is shown
Primary Actors	User
Secondary Actors	Database
Trigger	User selects an exercise-related action: Create Exercise, Edit Exercise, or Delete Exercise.
Main Flow	1. User selects Manage Exercises. 2. System displays options: Create, Edit, Delete. 3. User selects Create Exercise. a) System displays the exercise creation form. b) User enters name, muscle group, and optional notes. c) System validates the input. d) System stores the new exercise in the database. e) System confirms success and updates the exercise list. 4. User selects Edit Exercise. a) System displays the selected exercise's current details. b) User edits name, muscle group, or notes. c) System validates updated information. d) System stores the updated exercise in the database. e) System confirms success and updates the exercise list. 5. User selects Delete Exercise. a) System asks for confirmation. b) User confirms deletion. c) System deletes the exercise from the database. d) System confirms success and updates the exercise list.
Extensions	3a (Create/Edit): If required fields are empty, system highlights missing values. 3c (Create/Edit): If validation fails, system shows an error and returns to the form. 3d (Any action): If database operation fails, system shows an error message. 5c (Delete): If user cancels deletion, no changes occur.



Use Case template

Use Case Name	Log Workout
Related Requirements	System has to let users create and save workout plans using selected exercises from their exercise logs and system has to let users log completed workouts, including sets, reps, and weights
Goal in Context	Allow the user to record sets, reps, and weights completed for a wo
Preconditions	User has at least one saved workout
Successful End Condition	Workout log is saved successfully
Failed End Condition	Data not recorded because of validation or system error
Primary Actors	User
Secondary Actors	Database or Analytics Engir
Trigger	User selects a workout to log
Main Flow	1. User opens a saved workout 2. System shows the list of exercises 3. User inputs sets, reps, weights, and marks exercises as completed 4. User submits the workout log 5. System stores data and updates progress stats in app 6. Confirmation message displays
Extensions	3a. If user leaves fields blank then system highlight: the missing entries 5a. If system cannot save data then user receives ar error message





Figures 2.1-2.4: Designs from the initial project phase (first semester), including use case diagrams for managing exercises and logging workouts, a sequence diagram showing system interactions, and an architecture diagram showing the layered structure of the React frontend, FastAPI backend, and SQLite database.

3. Second Semester Results (Implementation and Testing)

During the second phase, I fully implemented the system and added several new features beyond the original plan.

Some of the key features include:

- Full CRUD functionality for workouts
- Automatic exercise classification into muscle groups
- Volume calculation using sets, reps, and weight
- Personal record tracking
- Streak and consistency tracking
- Progression charts to show performance over time
- Training insights based on workout data
- Dark and light mode support

Originally, the project was meant to focus on basic tracking, but I expanded it to include analytics and insights. This made the system more useful by actually helping users understand their training instead of just logging it.

4. System Implementation

The backend was built using FastAPI and is responsible for handling API requests, validation, classification, and database operations. SQLite is used to store all workout data, including timestamps and calculated attributes such as muscle group classification.

The frontend was developed using React and serves as the web-based interface for the system. It allows users to log workouts, view analytics, and interact with their data in real time. The frontend communicates with the backend through API calls.

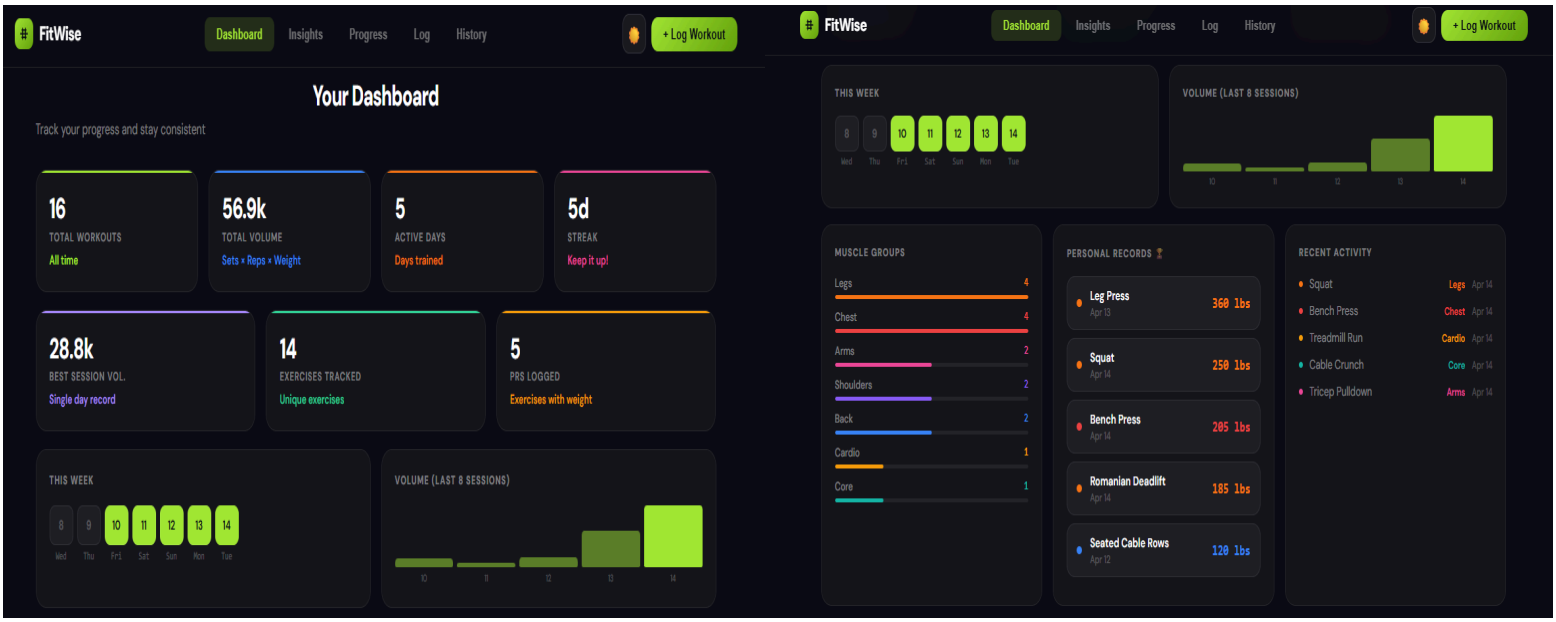


Figure 4.1 & 4.2: Dashboard displaying workout analytics including total volume, streaks, personal records, and muscle group distribution.

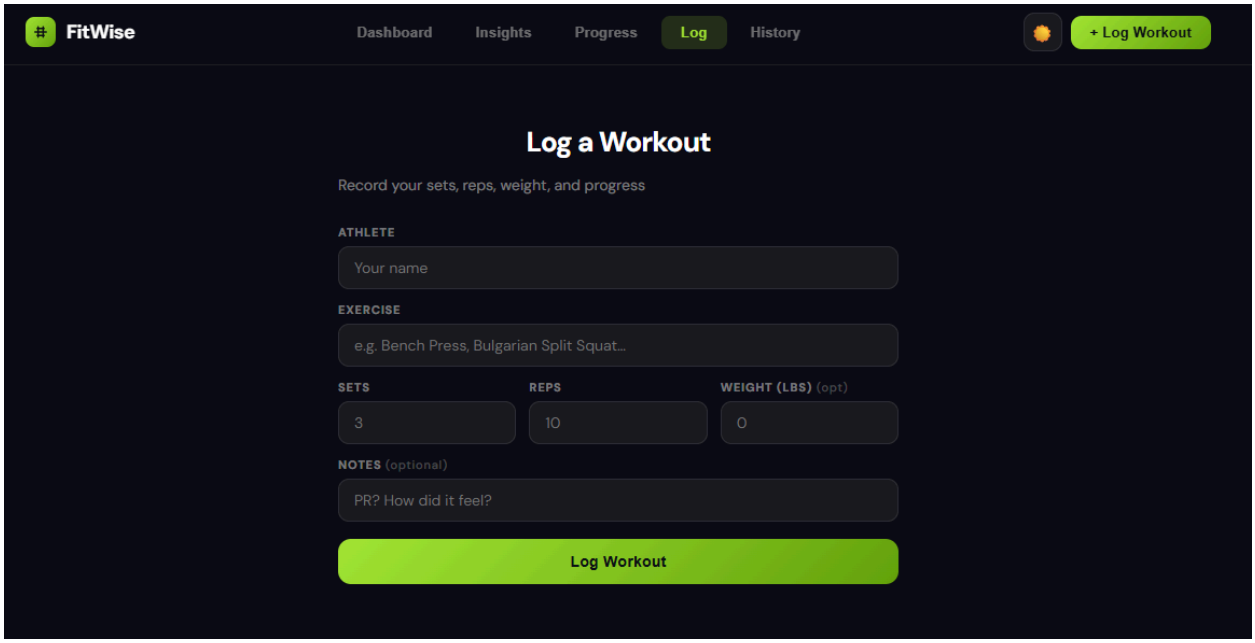


Figure 4.3: Workout logging interface where users input exercises, sets, reps, and weight with automatic classification into muscle groups.

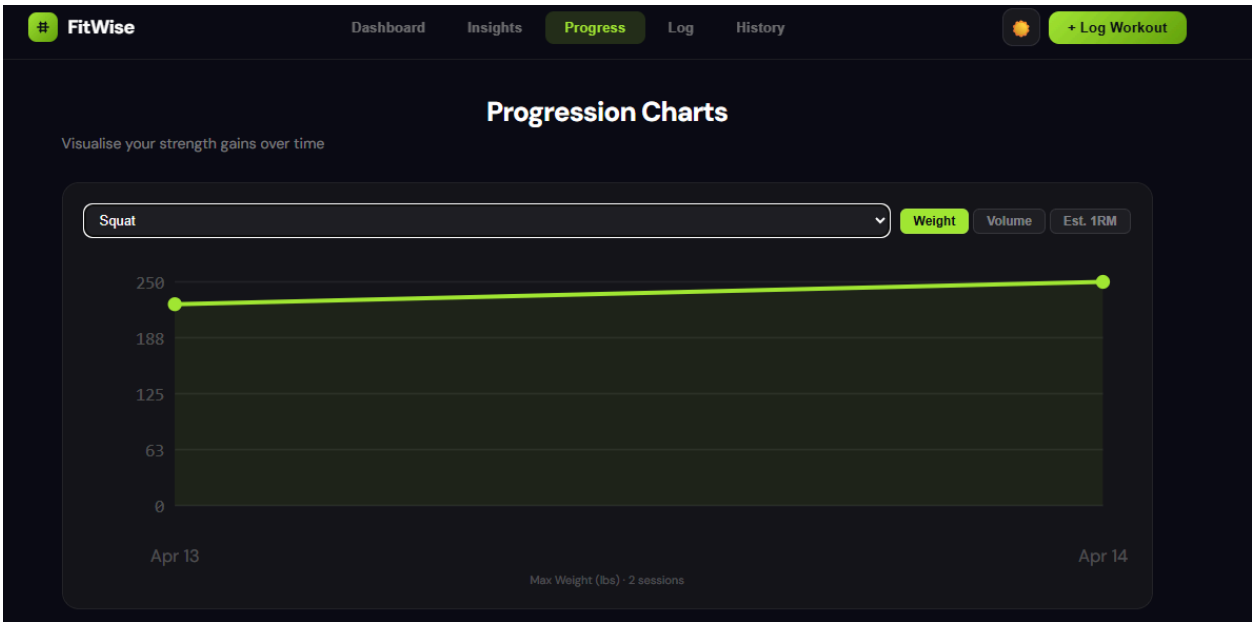


Figure 4.4: Visualization of workout progression over time / training insights generated from user data.

4.1 Tools and Techniques Used

The system was implemented using several tools and technologies. The backend was developed using FastAPI and run locally using Uvicorn. The frontend was built using React and executed using Vite.

Examples of commands used during development include:

- Starting backend server:
  `python -m uvicorn main:app --reload`
- Installing backend dependencies:
  `python -m pip install fastapi uvicorn`
- Running frontend development server:
  `npm run dev`

These tools allowed for rapid development, testing, and real-time updates during implementation.

5. Testing and Validation

I tested the system to ensure all CRUD operations work correctly, including creating, retrieving, updating, and deleting workouts.

Validation testing was performed using invalid inputs such as empty fields, negative values, and incorrect data types. The system correctly rejects invalid input and maintains data integrity.

I also tested the full workflow, from logging a workout to viewing updated analytics on the dashboard, to ensure the system operates as expected.

6. Final Results and Conclusions

The final system is fully functional and demonstrates a complete full-stack application with persistent data storage, real-time updates, and analytical features.

FitWise not only stores workout data but also analyzes it to provide insights into performance, consistency, and progression. For example, the system identifies personal records based on the highest weight logged per exercise and calculates training volume using sets × reps × weight.

This transforms the application from a simple tracking tool into a more intelligent system for training analysis.

7. Project Evolution

At the beginning, FitWise was designed as a simple system for organizing workout data.

During development, I expanded the project by adding features such as training insights, progression tracking, and automated classification. These additions changed the system from a basic tracker into a more advanced tool that can analyze workout data and provide more useful feedback.

8. Project Website

The React frontend serves as the web-based interface for the project. It allows users to interact with the system through a browser, log workouts, and view analytics, charts, and insights.

9. Analysis of Results

The system performs reliably for all core CRUD operations. Validation mechanisms make sure that invalid data is not accepted therefore improving the integrity of the data.

The integration between frontend and backend is successful, allowing real-time updates of workout data.

However, some limitations still exist:

- No authentication system is implemented
- The system does not support multiple users

Despite these limitations, the system meets all current functional requirements.

10. Conclusion

FitWise successfully evolved from a planned workout tracking system into a fully implemented full-stack application. The project demonstrates not only the ability to design a system, but also to build and refine it through multiple stages of development.

The final system supports workout logging, data persistence, and real-time analytics. More importantly, it extends beyond basic tracking by analyzing workout data to provide insights into performance, consistency, and progression.

This project highlights the transition from system design to full implementation, and shows how additional features such as classification, progression tracking, and training insights can significantly improve the usefulness of an application.

Overall, FitWise meets the project requirements and demonstrates a complete development process, from specification and design to implementation and testing.

11. User Manual

The FitWise system is designed to be simple and easy to use.

To use the system:

1. Open the web application in a browser
2. Navigate to the “Log Workout” page
3. Enter the exercise name, sets, reps, and weight
4. Submit the workout
5. View updated results on the dashboard

Users can also:

- View workout history
- Filter workouts by muscle group
- Analyze performance using charts and insights

The system automatically classifies exercises and updates analytics in real time.

12. Meeting Minutes

Project: FitWise Workout Tracking System
Date: April 14, 2026
Attendee: Emmanuel Dairo

April 1, 2026 – Planning and Review

- Reviewed Milestone #2 submission and feedback
- Identified areas for improvement including UI, analytics, and system depth
- Planned implementation of new features such as insights and progression tracking

April 5, 2026 – Backend Enhancements

- Improved data handling and validation

- Ensured consistent classification logic for exercises
- Verified database operations and API responses

April 8, 2026 – Frontend Development

- Redesigned user interface for better usability
- Added dashboard components for analytics
- Integrated frontend with backend API

April 10, 2026 – Feature Expansion

- Implemented progression charts for tracking performance over time
- Added training insights based on workout data
- Introduced dark and light mode functionality

April 13, 2026 – Testing and Refinement

- Tested full system workflow from input to analytics
- Fixed UI inconsistencies and minor bugs
- Verified accuracy of volume, PR tracking, and classification

April 14, 2026 – Final Preparation

- Finalized presentation and progress report
- Prepared demo flow
- Reviewed system functionality to ensure readiness for submission